ROTEIRO PRÁTICO DA DISCIPLINA

Curso *Engenharia de Software*	**Campus** *Rio Verde*
Disciplina *Banco de Dados I*	
Professor *Sergio Souza Novak*	**Turma** *A / B*
Código da Atividade *ESW417*	

Configuração Importante: Google Colab e SQLite

Para resolver este roteiro prático utilizando SQL no **Google Colab**, certifique-se de executar as etapas iniciais de instalação da extensão SQL e, **fundamentalmente**, habilitar a integridade referencial de chaves estrangeiras executando o seguinte comando no início do seu notebook:

```
%sql PRAGMA foreign_keys = ON;
```

Se essa diretiva não for executada, o SQLite aceitará inserções consistentes de chaves órfãs, prejudicando o teste das restrições DDL formuladas.

1 Passo 1: Preparando o Ambiente

Instale o pacote jupysql e configure a conexão com SQLite:

```
!pip install jupysql
%load_ext sql
%sql sqlite:///meu_banco.db
%sql PRAGMA foreign_keys = ON;
```

2 Passo 2: Modelagem das Tabelas

Crie as tabelas base:

```
CREATE TABLE Cursos(
  id INTEGER PRIMARY KEY,
  nome TEXT NOT NULL
);

CREATE TABLE Alunos(
  id INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  idade INTEGER NOT NULL,
  curso INTEGER,
  FOREIGN KEY (curso) REFERENCES Cursos(id)
);
```

3 Passo 3: Populando o Banco de Dados

Insira registros de cursos e alunos:

```
-- Cursos
INSERT INTO Cursos (nome) VALUES ('Engenharia de Software');
INSERT INTO Cursos (nome) VALUES ('Ciencia da Computacao');
INSERT INTO Cursos (nome) VALUES ('Analise e Desenvolvimento de Sistemas');
INSERT INTO Cursos (nome) VALUES ('Sistemas de Informacao');
INSERT INTO Cursos (nome) VALUES ('Redes de Computadores');

-- Alunos
INSERT INTO Alunos (nome, idade, curso) VALUES ('Joao Silva', 20, 1);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Maria Oliveira', 22, 2);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Pedro Santos', 19, 1);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Ana Souza', 21, 3);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Carlos Lima', 23, 2);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Julia Costa', 20, 4);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Rafael Martins', 24, 3);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Fernanda Almeida', 18, 5);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Gustavo Pereira', 21, 4);
INSERT INTO Alunos (nome, idade, curso) VALUES ('Isabela Rocha', 22, 5);
```

4 Passo 4: Praticando Window Functions

4.1 Agregacao Comum vs Window Function

```
-- Agrupamento com GROUP BY
SELECT curso, AVG(idade) FROM Alunos GROUP BY curso;
```

```
-- Window Function preservando linhas
SELECT nome, curso, AVG(idade) OVER (PARTITION BY curso) AS media_curso
FROM Alunos ORDER BY curso;
```

4.2 Acumulando Valores (Running Total)

```
SELECT nome, curso,
       SUM(idade) OVER (PARTITION BY curso ORDER BY nome DESC) AS somatorio
FROM Alunos ORDER BY nome DESC;
```

5 Passo 5: Praticando Views

Crie uma view que une alunos e cursos:

```
CREATE VIEW alunos_cursos AS
SELECT Alunos.id, Alunos.nome, Cursos.nome AS curso
FROM Alunos INNER JOIN Cursos ON Alunos.curso = Cursos.id;
```

Consultando a view:

```
SELECT * FROM alunos_cursos;
```

6 Passo 6: Praticando Triggers e Auditoria

6.1 Tabela de Log

```
CREATE TABLE log_alunos_cursos (
  log_id INTEGER PRIMARY KEY AUTOINCREMENT,
  aluno_id INTEGER NOT NULL,
  aluno_nome TEXT NOT NULL,
  curso_antigo_id INTEGER,
  curso_novo_id INTEGER,
  operation_type TEXT,
  data_modificacao DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

6.2 Triggers

```
-- UPDATE trigger
CREATE TRIGGER log_de_mudanca_cursos
AFTER UPDATE OF curso ON Alunos
FOR EACH ROW WHEN OLD.curso IS NOT NEW.curso
BEGIN
  INSERT INTO log_alunos_cursos (aluno_id, aluno_nome, curso_antigo_id,
   curso_novo_id, operation_type)
  VALUES (OLD.id, OLD.nome, OLD.curso, NEW.curso, 'UPDATE');
END;

-- INSERT trigger
CREATE TRIGGER log_de_insercao_alunos
AFTER INSERT ON Alunos
FOR EACH ROW
BEGIN
  INSERT INTO log_alunos_cursos (aluno_id, aluno_nome, curso_novo_id,
   operation_type)
  VALUES (NEW.id, NEW.nome, NEW.curso, 'INSERT');
END;

-- DELETE trigger
CREATE TRIGGER log_de_remocao_alunos
AFTER DELETE ON Alunos
FOR EACH ROW
BEGIN
  INSERT INTO log_alunos_cursos (aluno_id, aluno_nome, curso_antigo_id,
   operation_type)
  VALUES (OLD.id, OLD.nome, OLD.curso, 'DELETE');
END;
```

6.3 Teste dos Triggers

```
-- INSERT test
INSERT INTO Alunos (nome, idade, curso) VALUES ('Novo Aluno', 20, 1);
-- UPDATE test
UPDATE Alunos SET curso = 3 WHERE id = 1;
-- DELETE test
DELETE FROM Alunos WHERE nome = 'Novo Aluno';
```

Verifique o log:

```
SELECT * FROM log_alunos_cursos;
```